

Background

Context: Data privacy and control over ownership of stored data has become an increasingly important concern with recent issues of government or company over-reach. Although we may be unable to change the existing infrastructure in the field of cloud storage, this project aims to inject confidentiality through the creation of a distributed key-value store that splits each object value into multiple fragments across different nodes owned by different services/regions. Thus, as no single node stores the full object, and reconstruction requires a majority of containers in the system to coordinate, we reduce the ability of any individual region from unilaterally obtaining the full stored value.

General idea: At its core, our system involves a “master” metadata coordination server which is responsible for interfacing with the client and coordinating with our multiple “regional” key-value fragment stores. We do not design for full Byzantine fault tolerance, but do have some design choices that aim to deal with a subsection of malicious actors. Specifically, clients may issue malformed, replayed, or unauthorized requests through the public API, and regional stores may return stale or corrupted fragments, which the coordinator attempts to detect using metadata and integrity checks. Clients interact via standard HTTP requests (get/put/update), and are treated as untrusted actors that must authenticate before the plaintext data is returned. Rather than storing the full object in a singular region, we first encrypt the object and then apply a configurable (k, m) erasure-coding scheme. This will ensure we have k data fragments, and m parity fragments - meaning that any k valid fragments are sufficient to reconstruct the encrypted data. Furthermore, we do not store the decryption key in full in any single region; instead, it is split across regional key-share holders under a threshold policy. The key-reconstruction threshold may, but need not, match the data reconstruction threshold k . The former ensures data is not complete in any one region, the latter ensures the secrecy of that data.

The metadata coordinator acts as the control plane. It tracks object versions, erasure-coding configuration, fragment placement, integrity metadata (e.g. checksums), and commit status. The regional fragment stores are only responsible for durably storing the fragment they have received, and cannot themselves coordinate with other regions. The coordinator reconstructs the ciphertext from fragments, gathers enough valid key shares to decrypt it, and only then returns plaintext to the client.

Upon receiving a write request, the coordinator creates a new version for the key, encrypts the value, encodes fragments according to the erasure coding scheme, and sends the fragment to the assigned regional fragment stores. It also distributes the object/version specific corresponding key shares across the regional key-share holders. Regional stores persist their fragment and ack back to the coordinator upon success or failure. A write is committed only once the coordinator has received successful acknowledgements for at least k valid ciphertext fragments and successful placement of at least the threshold number of key shares required for later decryption. Upon receiving a read request, the coordinator authenticates the client, verifies authorization for the object, consults metadata for the latest committed version, fetches any k valid fragments to reconstruct the encrypted ciphertext, and then gathers enough key shares to decrypt it. We return the final reconstructed plain-text back to the client, failing if we are unable to retrieve the threshold of fragments or key-shares.

The system in general prioritizes strong consistency, partition tolerance, and cross-region confidentiality. Naturally, this means we have an associated tradeoff in terms of availability, latency, and work needed to process messages. These trade-offs are deliberate and expected, as our system is meant to durably store confidential information ... rather than low-latency, high-update workloads.

Failure Model: We focus our system on being tolerant to the “timing” and “crash” failure models. This is because in our system, it is impossible to distinguish between a region that has crashed from one that is temporarily unavailable or slow. Messages between different parts of our system including the coordinator, client, and regional servers, may be delayed, lost, duplicated, reordered, or prevented entirely via partitions. Thus, we bound our system on response timing through timeouts, retries, and liveness checks rather than assuming reliable communication. Further, we do not design for full byzantine fault tolerance; however the coordinator is able to verify request authorization, fragment integrity, version correctness, and commit status such that stale, corrupted, or incomplete responses can be rejected. Each regional service sends periodic heartbeats to confirm its liveness. As the system prioritizes consistency and partition tolerance over availability, reads and writes will fail if the coordinator cannot safely gather the required thresholds of fragments or key shares for a particular committed version of an object.

Assumptions:

- We assume the coordinator remains alive and trusted for the base design. Although coordinator replication could improve reliability, it is out of scope for the current project.
- We assume the system consists of a fixed set of regions. This is a consideration for the prototype and avoids encountering issues like late system joiners, and how we must distribute old fragments among them.
- Each region acts as one logical service: if either its fragment store or key-share holder fails, that region is treated as unavailable for that object version.
- Regional services are not fully byzantine, but the coordinator can detect stale or corrupted responses using metadata, checksums, and commit state.
- We assume the regions themselves are largely durable. We could introduce PAXOS in a per-region fashion to ensure fragment replication. However, that is out of the scope of our current project. If we lose data beyond the allowable threshold, we cannot recover.
 - Data in the regions themselves may be stored in volatile or disk memory, these are abstractions and the choice can affect latency but is not a primary concern.
- Regional services do not coordinate directly with each other.
- Clients are untrusted, they may issue malformed or unauthorized requests via public API.
- We require operations to appear linearizable at the coordinator, even if requests arrive concurrently.
- Coordinator, client, and region communication are done in authenticated channels.

Desired Properties:

- No single region stores enough information to reconstruct the ciphertext and decrypt it to form the original object.
- The coordinator will only expose committed object versions.
 - A committed object must be reconstructable and decryptable.
- Fragment and key shares are version/object specific.
- Message and time handlers should be deterministic.
- Service should be consistent.
- Service should be resilient to network partitions.
- Operations should execute linearizably, that is one-at-a-time we can order them sequentially and reconstruct the state.
- Operations should provide exactly-once semantics, that is repeated instructions (perhaps from a duplicate request) do not cause changes in the system.
- The erasure coding scheme and placement policy is configurable.

- Incomplete writes must not become visible and may be safely garbage-collected.
- The coordinator can detect and reject malformed fragment responses.
- Only authenticated and authorized clients may cause plaintext release or object updates.
- Replayed client write requests must be detected and rejected or deduplicated.
- Communication between the coordinator and regional services occurs over authenticated and integrity-protected channels.
 - This could be as simple as using TLS over the channels linking each entity.

Deduced Invariants:

- A specific object version becomes visible only after the coordinator has recorded it as committed.
- A committed object version has at least the minimum required threshold of valid ciphertext persisted.
- A committed object version has at least the required threshold of valid key shares persisted.
- Every successful read reconstructs from fragments belonging to one committed version only.
- No single region can reconstruct ciphertext and decrypt it alone.
- Fragments and key shares used in reconstruction must match the committed version metadata.
- Each client request changes the store at most once.
- If the coordinator remains alive, and enough regions are reachable to satisfy both thresholds, retries eventually allow the operations to complete.

Protocol Design

We will discuss the happy paths of our design, its initialization, and failure cases.

Happy Write Path:

1. Client sends an authenticated put or update request to the coordinator, including (clientId, requestId) for deduplication.
2. Coordinator verifies authentication and authorization for the requested object.
3. Coordinator creates a new object version v .
4. Coordinator generates a fresh per-version data-encryption key and encrypts the value.
5. Coordinator applies the configured erasure-coding scheme (k, m) to the ciphertext, producing $k + m$ fragments.
6. Coordinator splits the decryption key into threshold key shares and assigns one share to each participating region.
7. Coordinator sends each ciphertext fragment and corresponding metadata to its assigned regional service, and sends the corresponding key share to that region's key-share holder.
8. Each regional service durably stores its fragment and key share, then acknowledges success or failure.
9. Once the coordinator has received successful acknowledgements from at least the required threshold of regions for both fragment persistence and key-share persistence, it marks version v as committed in metadata.
10. Coordinator returns success to the client

Happy Read Path:

1. Client sends an authenticated get request to the coordinator.
2. Coordinator verifies authentication and authorization for plaintext release.
3. Coordinator looks up the latest committed version v for the requested object.
4. Coordinator requests any sufficient threshold of valid ciphertext fragments for version v from the regions.
5. Coordinator verifies fragment integrity and reconstructs the ciphertext.
6. Coordinator requests the required threshold of valid key shares for version v .
7. Coordinator reconstructs the decryption key and decrypts the ciphertext.
8. Coordinator returns the plaintext object to the client.

In the background, each regional service sends heartbeats to the coordinator every fixed interval. The coordinator uses these heartbeats to maintain a liveness view of the regions. All request/response steps that expect a reply are governed by timeouts and retries with bounded backoff.

Initialization:

At startup, the coordinator is configured with:

- the fixed set of participating regions
- the erasure-coding configuration (k, m)
- the key-share threshold policy
- the placement policy for fragments and key shares

Each regional service registers with the coordinator and begins sending periodic heartbeats. The coordinator records which regions appear alive and available. Once enough regions are available to satisfy both the fragment reconstruction threshold and the key-share threshold, the system is considered initialized and capable of serving requests.

Clients communicate only with the coordinator and do not need direct knowledge of regional addresses. Clients authenticate against the coordinator before issuing reads or writes.

Failure Cases Communication:

- Client:
 - Fails to receive any response from the server it contacts.
 - Solution: Client retries the request with exponential backoff. It resets its interval upon receiving the response, and continues to cycle.
 - Receives Failure Response:
 - Operation Rejected:
 - Solution: No issue here, this can happen if the API request was invalid.
 - Authentication/Authorization error:
 - The request is rejected because the client either is not communicating via an authenticated channel, or is not authorized to gain info about that specific object.
 - Solution: can only join channel, authorization cannot be changed after storage.
 - Busy:
 - Depending on the decided complexity, it could be the case that the server is currently busy responding to another request.

- Solution: We have to decide whether to buffer the requests on the server side or reject outright. I suggest buffering client requests, we will still respond busy when we reach a set limit. Thus the client should retry, and the server can be responsible for deduplication.
- Corruption Error:
 - This response will be received if there is some inconsistency in the system that prevents the server from executing the request permanently, for example we have gone below the k fragments needed for reconstruction for a particular object, the server will not retry. This is unlikely to be sent, because we do not have “permanent” failures/crashes being distinguished in our model. This could be a consideration for a real system though.
- Coordinator (Writes):
 - Fails to receive enough fragment acknowledgements:
 - Solution: The coordinator retries the missing regional writes with backoff. Whenever we receive k valid fragment acknowledgements we commit, if we exceed a certain threshold of retries we may instead decide to return a failure response to the client to avoid blocking the system.
 - Fails to receive enough key-share acknowledgements:
 - Solution: The coordinator retries the missing regional key share writes with backoff. Whenever we receive k valid key share acknowledgements we commit, if we exceed a certain threshold of retries we may instead decide to return a failure response to the client to avoid blocking the system.
 - Note, both the key-share and fragment acknowledgements must occur for a particular region. If either fails, then for that region the commit ack should not be returned.
 - Receives stale, corrupted, or mismatched fragment/key-share response:
 - Solution: the coordinator rejects the response using version metadata, checksums, and commit state, and does not count that region toward required threshold.
 - Receives duplicate request (clientId, requestId):
 - Solution: we choose to buffer requests, if the same request is repeatedly received from a client we can assume it is because of issues with message delivery. We return the previously recorded result.
- Coordinator (Reads):
 - Fails to retrieve enough valid ciphertext fragments:
 - Solution: The coordinator retries alternate valid region combinations. If no set of k valid fragments can be obtained, the read fails
 - Fails to retrieve enough valid key shares:
 - Solution: The coordinator retries alternate valid region combinations. If no set of k valid key shares can be obtained, the read fails.
 - Receives stale, corrupted, or mismatched fragment / key-share response:
 - Solution: The coordinator rejects the response and continues only if another valid threshold set can still be formed.
- Regional Service
 - Receives duplicate write for same object version:
 - Service rejects the request.
 - Receive request for stale version:
 - Reject request

- Misses heartbeat interval:
 - Coordinator marks this region as suspected failed, and does not waste requests on this service.

Other Failure Cases:

- Region Failure:
 - A region may crash or become unreachable. Since each region acts as one logical service, both its fragment store and key-share holder are treated as unavailable.
 - If enough remaining regions exist to satisfy both fragment and key-share thresholds, the system may continue to serve operations. If fewer than k regions remain reachable for either fragments or key shares, reads and writes fail.
- Partial Write:
 - Some regions may successfully persist fragments or key shares for a new version while others fail.
 - Solution: The coordinator does not commit that version. Uncommitted fragments and key shares are ignored by reads and may later be garbage-collected.
- Network Partition:
 - Some regions may be partitioned away from the coordinator.
 - Solution: If the coordinator cannot gather at least k valid fragments and k valid key shares, the operation fails. We sacrifice availability here to preserve consistency and prevent committed but unrecoverable versions.

Observability Plan

We will log lightweight **request, reconstruction, authorization, and regional health metrics** so that we can debug correctness, confidentiality violations, and performance bottlenecks. In particular, we want to observe: whether reads/writes are succeeding, whether versions are being committed correctly, whether fragment/key-share thresholds are being met, whether regions are becoming unavailable, and whether unauthorized or replayed requests are being rejected. We will also track latency across the major stages of the read and write paths so we can identify whether slowdowns come from encoding, regional persistence, reconstruction, key-share recovery, or coordinator-side verification.

Examples:

- client ops: (clientId, reqId, op=get|put|update, objectKey, version?, outcome=success|reject|timeout|retry, latencyMs)
- auth / authorization checks: (clientId, reqId, objectKey, op, authResult=success|reject, reason)
- replay detections: (clientId, reqId, objectKey, outcome=rejected|deduplicated)
- version commit attempts: (objectKey, version, fragmentAcks, keyShareAcks, committed=true|false)
- fragment writes: (objectKey, version, region, outcome=success|fail|timeout, latencyMs)

- key-share writes: (objectKey, version, region, outcome=success|fail|timeout, latencyMs)
- read reconstruction: (objectKey, version, fragmentsRequested, fragmentsUsed, outcome=success|fail, latencyMs)
- key reconstruction: (objectKey, version, keySharesRequested, keySharesUsed, outcome=success|fail, latencyMs)
- plaintext release: (clientId, objectKey, version, authorizationResult, outcome=released|denied)
- heartbeat / liveness: (region, heartbeatSeen, status=alive|suspected-failed|recovered)
- stale / corrupted responses: (region, objectKey, version, responseType=fragment|keyShare, reason=checksumMismatch|versionMismatch|incomplete)
- threshold failures: (objectKey, version, availableFragments, availableKeyShares, requiredK, opType=read|write)
- garbage collection of uncommitted state: (objectKey, version, cleanedFragments, cleanedKeyShares)

Other Ideas

- Can we compress the store at some point to only keep latest version of objects?
- Should we employ replication at the coordinator level to help in case of reliability issues (Primary-backup?). What about PAXOS at the regional level?
- Should we consider repairs, or dynamic region membership. For example we want to replace a failed region so we reconstruct the objects stored and then populate the new region?
- Is this design too simple? What are we not considering?